

Smart SSH Terminal

Andres Felipe Gonzalez Garzon, Haider Andres Cañon Murcia

Bogotá, Colombia

andres-gonzalez16@upc.edu.co, Haider-canon@upc.edu.co

Resumen—A lo largo de la historia de internet, los sistemas operativos han jugado un papel fundamental. En particular, los sistemas basados en Unix, como Linux, constituyen la columna vertebral de la infraestructura que sostiene la red global. Su administración se realiza principalmente a través de la terminal de línea de comandos. Este paper propone una solución innovadora que integra un modelo de lenguaje grande (LLM) en un cliente SSH, facilitando la interacción y el aprendizaje en el entorno de la terminal.

Index Terms—SSH, Linux, LLM, Terminal Inteligente, cliente SSH, Administración de Sistemas, Asistente de IA, Unix/Linux

I. INTRODUCCIÓN

Desde los orígenes de la computación moderna, los sistemas operativos han sido el puente entre el ser humano y la máquina. Sin importar la época ni el fabricante, todos ellos han compartido un rasgo común: una interfaz de comandos textual que, si bien otorga un control preciso y profundo sobre el sistema, exige al usuario memorizar una sintaxis extensa, interpretar mensajes de error crípticos y construir mentalmente un modelo abstracto de cómo funciona el sistema [1]. Este escenario, que décadas atrás era exclusivo de especialistas, sigue siendo hoy el punto de entrada obligado para cualquier persona que aspire a comprender y gestionar sistemas operativos basados en Unix/Linux [2]. El resultado es documentado empíricamente: los mensajes de error de las herramientas de línea de comandos presentan una dificultad sustancial para los usuarios novatos [3]; esta dificultad desencadena ciclos de frustración [4] que, si se repiten sin retroalimentación efectiva, producen desenganche emocional progresivo de la tarea —patrón documentado en novatos de programación ante errores repetidos [5] y estructuralmente análogo al aprendizaje de entornos CLI, donde la misma ausencia de retroalimentación inmediata opera como factor de abandono.

El auge reciente de los Modelos de Lenguaje Grande (LLMs) representa un punto de inflexión en este panorama. Por primera vez, es posible interactuar con sistemas complejos empleando lenguaje natural, recibir explicaciones contextuales al instante y obtener asistencia en la resolución de errores sin abandonar el entorno de trabajo [6], [7]. Esta capacidad abre una oportunidad sin precedentes para transformar la forma en que los estudiantes y técnicos aprenden a operar un sistema operativo: no a través de la memorización pasiva, sino mediante la práctica guiada, contextual y progresiva [8]. Sin embargo, en la práctica actual, la asistencia de IA permanece desacoplada del entorno donde ocurre el aprendizaje: el estudiante interrumpe su sesión de terminal, consulta un chatbot en el navegador y regresa al contexto de trabajo habiendo perdido

tanto el flujo cognitivo como la trazabilidad del error. Para que el potencial pedagógico de los LLMs se materialice, la inteligencia artificial debe integrarse de forma nativa en el mismo espacio donde ocurre la práctica: la terminal remota.

Precisamente de esa necesidad nace *Smart SSH Terminal*: un entorno de trabajo integrado que fusiona en una sola interfaz el acceso remoto seguro vía SSH, la gestión de archivos por SFTP y un asistente de IA contextual que acompaña al usuario en tiempo real. La herramienta está dirigida tanto a estudiantes de ingeniería como a técnicos en formación que requieren un andamiaje inteligente para desarrollar competencias reales en el manejo de Linux en entornos de laboratorio remoto con recursos limitados. Estos requisitos de eficiencia de memoria, bajo consumo de CPU y seguridad en las comunicaciones motivaron la elección de Rust y el framework Tauri como base tecnológica [9].

Las soluciones existentes abordan estos problemas de forma parcial: los clientes SSH tradicionales carecen de componente pedagógico; las plataformas de aprendizaje interactivo de Linux no ofrecen acceso real a infraestructura remota; y los terminales modernos con IA como Warp presentan limitaciones funcionales sobre sesiones SSH remotas y no incorporan mecanismos de andamiaje progresivo [10]. Ninguna herramienta integra simultáneamente acceso remoto autenticado, gestión de archivos y asistencia de IA pedagógica contextual en un único entorno. La Sección II documenta este vacío en detalle.

Las contribuciones principales de este trabajo son: (1) el diseño e implementación de *Smart SSH Terminal*, un cliente SSH con asistente de IA integrado de forma nativa que transforma los errores de ejecución en retroalimentación pedagógica contextual; (2) una arquitectura eficiente en Rust y Tauri que satisface los requisitos de cómputo de servidores de laboratorio de bajo coste; y (3) una evaluación empírica del sistema sobre infraestructura real, con métricas de rendimiento (uso de CPU, memoria y latencia de respuesta).

El resto del artículo se organiza así: la Sección II revisa los trabajos relacionados; la Sección III describe la arquitectura del sistema y sus subsistemas; la Sección IV justifica las decisiones de implementación, en particular la adopción de Rust; la Sección V presenta la metodología de pruebas y los resultados en el laboratorio remoto; finalmente, las Secciones VI y VII abordan el trabajo futuro y las conclusiones.

II. DISEÑO ARQUITECTÓNICO

II-A. Arquitectura General: Topología de Red y Subsistemas Lógicos

En un enfoque alineado con las arquitecturas de software modernas, el sistema ha sido concebido bajo un modelo encapsulado en el que la totalidad del aplicativo reside en la estación de usuario (Smart Client). Este diseño exime la necesidad de desplegar servidores propietarios intermedios; en su lugar, la lógica recae sobre una aplicación cliente autónoma que se acopla directamente a entidades externas y estandarizadas (como placas de desarrollo IoT o servidores Linux corporativos).

Como se ilustra en la Figura 1, la topología global describe una única **Aplicación Local** que emplea protocolos seguros para orquestar la interacción técnica con: (1) el **Laboratorio Físico** (ej. dispositivos Raspberry Pi mediante servicios nativos), (2) **Servicios Semánticos** (Plataformas LLM en la nube o locales) y (3) Servicios de Directorio Institucional (**LDAP**).

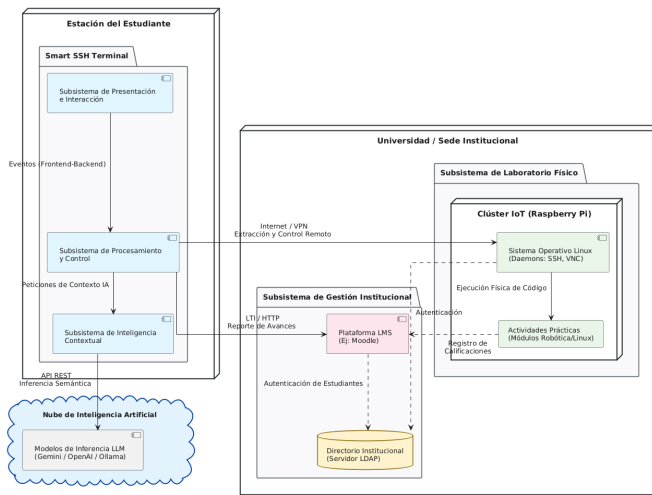


Figura 1. Topología general del sistema que demuestra la autonomía de la aplicación local frente a integraciones externas, prescindiendo de servidores intermedios propios.

A nivel interno, la aplicación cliente se divide en tres subsistemas lógicos fundamentales comunicadas asincrónicamente mediante un canal de paso de mensajes (IPC, de sus siglas en inglés), lo cual garantiza alta resiliencia y el aislamiento de las operaciones de red.

II-A1. Subsistema de Presentación e Interacción: Representa la superficie de control de cara al usuario. Su objetivo es unificar múltiples canales de administración remota de forma visual y reactiva (ver Figura 2). Este subsistema se compone de módulos funcionales interdependientes que operan bajo un gestor de estado centralizado:

- **Emulador de Terminal:** Procesa el flujo estándar de entrada y salida (I/O) emulando secuencias de control para soportar interactividad basada en texto de alta densidad.
- **Explorador de Archivos y Paneles de Control:** Traducción gráfica que transforma acciones de alto nivel (como

clicks sobre visores VNC) en operaciones de red directas al flujo del sistema operativo remoto.

- **Interfaz de Asistencia Conversacional:** Elemento de retroalimentación pedagógica, que sincroniza su contenido bidireccionalmente con las salidas de error o bloqueos presentados en el emulador de terminal.

Figura 2. Diagrama estructural del Subsistema de Presentación e Interacción.

II-A2. Subsistema de Procesamiento y Control (Core): Actúa como el motor de red y de sesión en el que se delegan las operaciones críticas. Como se indica en la Figura 3, este subsistema administra los algoritmos criptográficos, protocolos robustos de transporte y la auditoría de uso:

- **Motor de Canal Seguro:** Implementa el estándar SSH y sus negociaciones seguras (cifrado asimétrico), operando sobre descriptors para mantener multiplexados de forma asíncrona un túnel de terminal y flujos de red.
- **Motor de Transferencia de Archivos:** Encargado de empaquetar, enrutar y desempaquetar ráfagas binarias hacia el daemon SFTP o equivalentemente integrado en la conectividad del laboratorio remoto.
- **Orquestador de Trazabilidad:** Mecanismo que captura heurísticamente trazas de ejecución en la sesión, brindando al sistema una bitácora altamente estructurada de cada paso del operario.

Figura 3. Módulos transaccionales y de conectividad del Subsistema de Procesamiento Interno.

II-A3. Subsistema de Inteligencia Artificial Contextual: Este subsistema, ilustrado en la Figura 4, funge como puente middleware entre la estación cliente local y la red de transformadores y LLMs proveedores de respuestas, habilitando las capacidades de andamiaje:

- **Descubrimiento de Recursos (Context Protocol):** Dotación técnica que rompe el paradigma aislado de los chatbots, permitiendo a la IA inspeccionar el estado actual (archivos, logs del host remoto) como suplemento antes de retornar respuestas y deduciendo así el estado exacto del entorno con extrema fidelidad técnica.
- **Construcción de Contexto Pragmático:** Submódulo que enhebra un grafo con las salidas de error previas y con los comandos que generaron dichas anomalías de red o fallo general.
- **Motor Lógico de Prácticas Guiadas:** Autómata evaluador que sincroniza las habilidades demostradas en los comandos de sesión con escenarios metodológicos pautados de laboratorio (fases), personalizando la ayuda a la meta de aprendizaje.

Figura 4. Capa Semántica responsable de la coordinación de contexto hacia endpoints LLM.

II-B. Integración Arquitectónica y Beneficios Subyacentes

La topología expuesta erradica la fragmentación típica encontrada en clientes remotos [?]. La consolidación de canales (como conexiones SSH reusadas y WebSockets noVNC para escritorios virtuales) fusionados con un servicio *endpoint* inteligente reduce de forma abismal la latencia cognitiva de transición. El estudiante experimenta un *sandbox* único en donde errores operacionales son inmediatamente atrapados y digeridos semánticamente para proponer un apoyo educativo ininterrumpido.

III. IMPLEMENTACIÓN

III-A. Evolución del desarrollo

El desarrollo de esta herramienta comenzó inicialmente utilizando Tkinter, debido a que representaba la vía más rápida para prototipar una interfaz gráfica básica. Sin embargo, durante esta etapa se identificaron problemas críticos relacionados con el refresco de pantalla y la fluidez necesaria para un emulador de terminal en tiempo real.

III-B. Justificación de Rust

Ante las limitaciones de rendimiento encontradas, se decidió realizar un salto tecnológico hacia Rust [9]. Este lenguaje fue elegido por su enfoque en la seguridad de memoria [?], su alto rendimiento comparable al de C/C++ [?], y su ecosistema moderno que facilita la creación de aplicaciones de escritorio seguras. Rust permite una gestión eficiente de hilos y una respuesta inmediata en la interfaz de usuario.

III-C. Integración de servicios

La aplicación actual ya incorpora servicios equivalentes a los ofrecidos por herramientas tradicionales como WinSCP [?], permitiendo la gestión gráfica de archivos mediante el protocolo SFTP de forma integrada con la terminal.

III-D. La Novedad

La principal innovación radica en haber logrado una interfaz limpia y amigable que no solo emula una terminal, sino que es capaz de realizar un seguimiento inteligente a las acciones del usuario, ofreciendo asistencia contextual proactiva sin sobrecargar la experiencia de uso.

IV. ESCENARIO DE PRUEBAS

IV-A. Metodología

Para validar la herramienta, se llevó a cabo una metodología de pruebas con un grupo de 10 personas con distintos niveles de experiencia en Linux. El objetivo fue evaluar la facilidad de uso y la efectividad de la asistencia por IA.

IV-B. Métricas y gráficas

Los resultados mostraron tiempos de uso promedio de 20 minutos de interacción continua, durante los cuales los usuarios lograron generar comandos complejos y realizar tareas de administración que normalmente requerirían consultas externas a la documentación.

IV-C. Laboratorio remoto

La interfaz permite acoplarse a un escenario de laboratorio remoto [?], como el uso de una Raspberry Pi [?]. Esto permite a los usuarios desarrollar prácticas reales mientras monitorean el entorno físico. A continuación se presentan fotografías de evidencia del funcionamiento en este entorno.

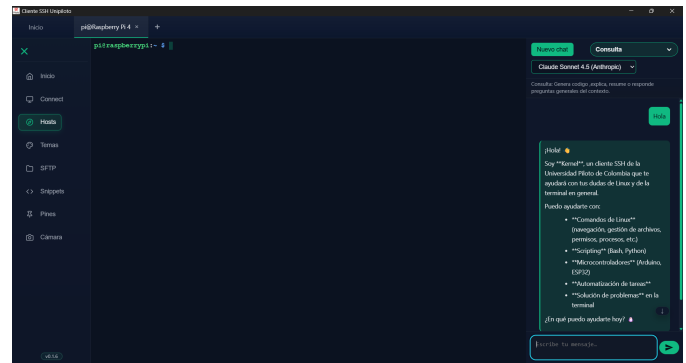


Figura 5. Interfaz del programa en ejecución.

V. TRABAJO FUTURO

Como líneas de trabajo futuro, se planea profundizar en la integración con modelos de IA, permitiendo al usuario elegir entre APIs comerciales (como ChatGPT de OpenAI o Gemini de Google) y soluciones de IA local (como Mistral) para garantizar la privacidad de los datos y evitar la sobrecarga del software con cálculos pesados en la nube.

Además, se propone añadir un botón de cambio rápido de idioma (español a inglés) directamente en la interfaz, facilitando su uso en entornos internacionales y bilingües.

AGRADECIMIENTOS

Se agradece a la Universidad, a los semilleros de investigación y a todos los integrantes del proyecto por el apoyo institucional y técnico brindado para la realización de este trabajo.

VI. CONCLUSIONES

El desarrollo de esta interfaz en Rust ha demostrado ser un éxito, proporcionando una herramienta robusta, segura y amigable para la administración de sistemas. El paper aporta una visión moderna de cómo la IA puede revitalizar el uso de la terminal.

Finalmente, las conclusiones se apoyan en la literatura existente que valida que "Linux y Rust van muy de la mano" [?], destacando la creciente adopción de Rust dentro del propio kernel de Linux y su relevancia en el desarrollo de herramientas de sistema de próxima generación.

REFERENCIAS

- [1] B. Shneiderman, "The future of interactive systems and the emergence of direct manipulation," *Behaviour & Information Technology*, vol. 1, no. 3, pp. 237–256, 1982.
- [2] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974.

- [3] B. A. Becker, P. Denny, R. Pettit, D. Bouchard, D. J. Bouvier, B. Harrington, A. Kamil, A. Karkare, C. McDonald, P.-M. Osera, J. L. Pearce, and J. Prather, "Compiler error messages considered unhelpful: The landscape of text-based programming error message research," in *Proceedings of the Working Group Reports on Innovation and Technology in Computer Science Education (ITiCSE-WGR '19)*. ACM, 2019, pp. 177–210.
- [4] E. Ossovski and M. Brinkmeier, "Frustration tolerance among computer-science-related novice university students," *European Journal of Engineering Education*, vol. 49, no. 4, pp. 734–751, 2024.
- [5] N. Bosch and S. D'Mello, "The affective experience of novice computer programmers," *International Journal of Artificial Intelligence in Education*, vol. 27, no. 1, pp. 181–206, 2017.
- [6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [7] M. Chen *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
- [8] P. Denny, V. Kumar, and N. Giacaman, "Conversing with Copilot: Exploring prompt engineering for solving CS1 problems using natural language," in *Proceedings of the 54th ACM Technical Symposium on Computer Science Education (SIGCSE 2023)*. ACM, 2023, pp. 1136–1142.
- [9] S. Klabnik and C. Nichols, *The Rust Programming Language*. No Starch Press, 2019.
- [10] Warp Technologies, "Feature support over SSH," <https://docs.warp.dev/code/ssh-feature-support>, 2024, accessed: 2025.